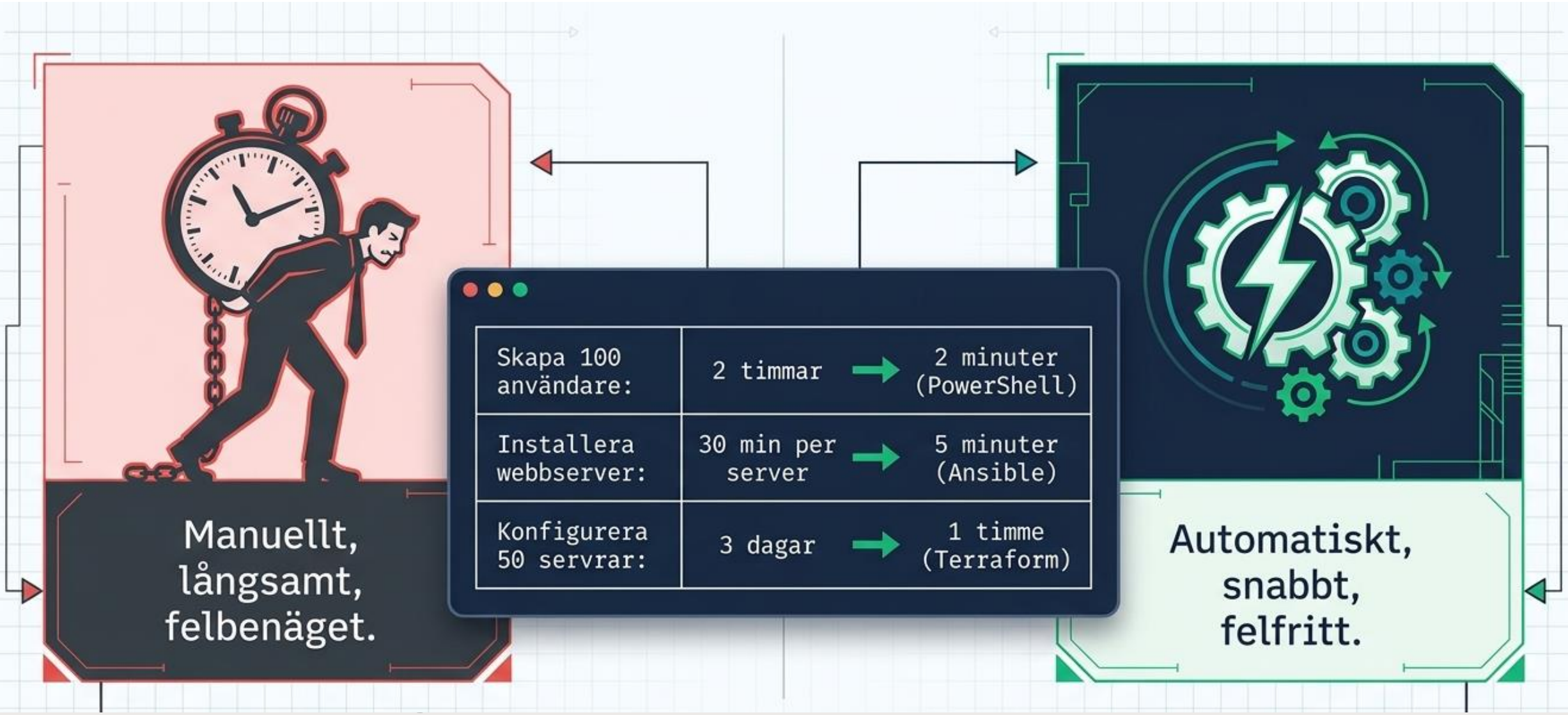
1. Kursintro & Scripting grunder

Arseniy Buzyan



Fakta: Enligt jobbannonser från 2026 söker svenska företag aktivt efter Automation Engineers med kunskaper i PowerShell, Terraform och Ansible.

Workshop – "Mänsklig Dator"



Mål: Förstå hur viktigt det är med TYDLIGA instruktioner (som i ett skript).

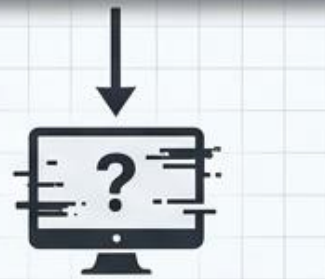


Instruktioner för läraren: Rita en cirkel på tavlan. "Jag är datorer. Ni ger er en instruktion, jag utför den EXAKT som ni säger."

Hur datorer tolkar instruktioner

Manuellt, otydligt, felbenäget

"Rita en cirkel på tavlan."





Otydlighet = Fel.

Automatiskt, exakt, framgångsrikt

1. Gå till tavlan.
2. Ta en röd penna.
3. Börja vid mittpunkten.
4. Rita en perfekt cirkel med 20 cm radie.





Precision = Succé.

> Datorer är dumma – de gör EXAKT vad du säger, varken mer eller mindre.

> Ett skript är inte magi. Det är en extremt precis lista med instruktioner. Automatisering bygger på att ge datorn en perfekt uppsättning regler att följa om och om igen.

Varför denna workshop?

Insikt

Datorer är dumma – de gör EXAKT vad du säger, inget mer.

Ett skript är en lista med EXAKTA instruktioner.

Automatisering = att ge datorn en PERFEKT lista med instruktioner.

Koppling till kursen

Skript måste vara PRECISA. En otydlig instruktion = fel!

Allt vi skriver i PowerShell, Terraform och Ansible är EXAKTA instruktioner.

Då slipper vi göra samma tråkiga uppgift om och om igen.

Vad Kan Du Automatisera?

Uppgift	Manuellt	Automatiserat
Skapa 100 användare	2 timmar	2 minuter med PowerShell
Installera webbserver	30 minuter per server	5 minuter med Ansible
Backa upp servrar	1 timme per dag	Körs automatiskt varje natt
Konfigurera 50 servrar	3 dagar	1 timme med Terraform

Enligt jobbannonser i Sverige: Automationskompetens (PowerShell, Terraform, Ansible) är bland de mest efterfrågade färdigheterna för IT-infrastrukturspecialister.

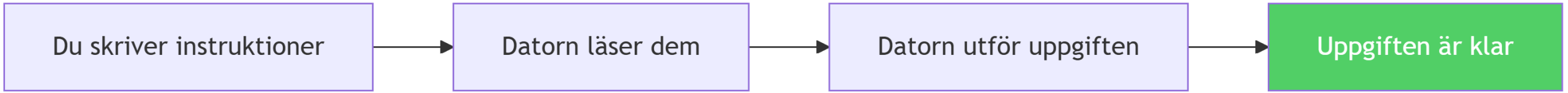
Vad är Ett Skript?



Ett skript är en textfil med instruktioner som datorn utför i ordning.



Exempel: Ett recept för att baka en kaka är som ett skript. Du följer stegen i ordning, och resultatet blir en kaka!



Varför Skript?

Fördel	Förklaring
Snabbt	Datorn gör samma sak varje gång, mycket snabbare än en människa
Felfritt	Inga stavfel eller glömda steg
Återanvändbart	Samma skript kan köras om och om igen
Spårbart	Alla steg loggas, så du vet vad som hände

Vad Kan Man Göra Med Skript?

- **Administration:**
- - Starta/stoppa tjänster och processer
- - Skapa, ändra eller ta bort användare
- - Hantera filer och mappar
- - Läsa och skriva till registret (Windows)
- **Övervakning:**
- - Kontrollera att servrar är igång
- - Skicka larm vid problem
- - Logga alla händelser
- **Automatisering:**
- - Installera programvara automatiskt
- - Konfigurera servrar
- - Bygga hela IT-miljöer med kod

Från isolerade uppgifter till fullständigt automatiserade IT-miljöer.



Kursens Syfte – Varför Just Denna Kurs?



Kursens Syfte – Varför Just Denna Kurs?



Du är i slutet av din utbildning och ska nu:



Samla ihop alla dina kunskaper



Lära dig automatisera på riktigt



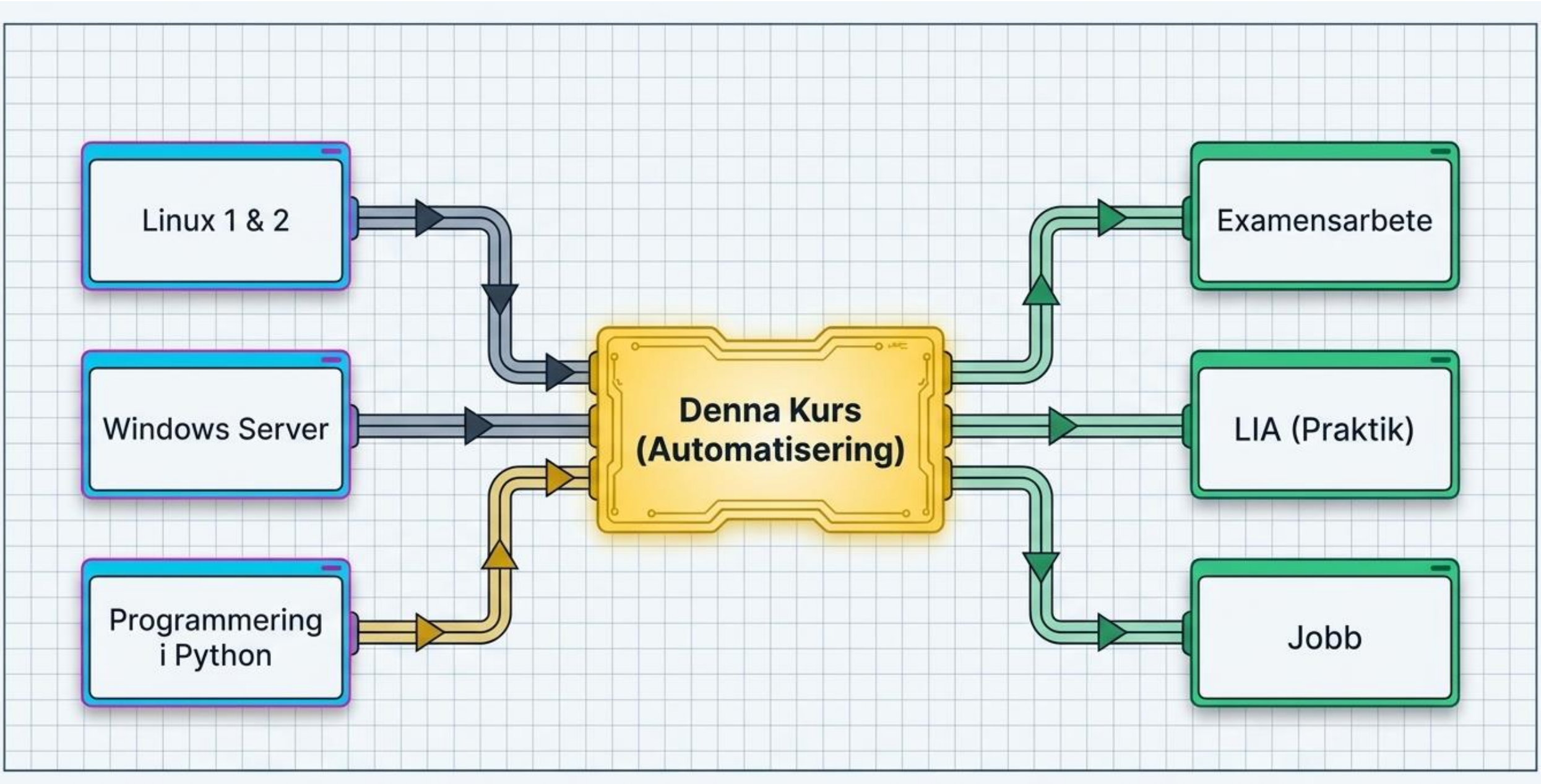
Förbereda dig för LIA och arbetslivet

Varför är detta viktigt?

Utan automatisering	**Med automatisering**
Mycket manuellt arbete	Allt görs automatiskt
Hög risk för fel	Färre mänskliga fel
Långsam driftsättning	Snabb och effektiv
Svårt att skala upp	Kan hantera tusentals servrar

Efter kursen ska du:

- Skriva PowerShell-skript för att automatisera Windows-administration
- Använda Bash (grundläggande) för Linux i WSL2
- Skriva Python-skript för att verifiera och orkestrera
- Använda Terraform för Infrastructure as Code (IaC)
- Använda Ansible för konfigurationshantering
- Bygga skriptkedjor där flera verktyg samverkar


Linux 1 & 2
Windows Server
Programmering i Python
Denna Kurs (Automatisering)
Examensarbete
LIA (Praktik)
Jobb

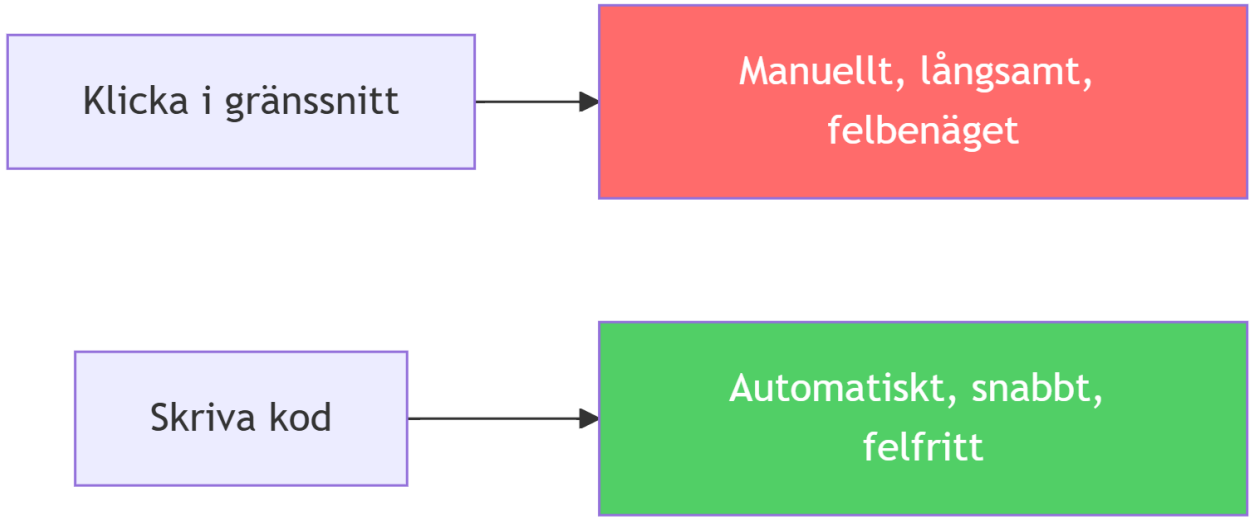
Tidigare kurs	Används i denna kurs
Linux 1 & 2	WSL2, Bash, Ansible
Windows Server	PowerShell, DSC, IIS
Programmering i Python	Python-skript, automatisering

Vad Är Infrastructure as Code (IaC)?

Definition: Att beskriva din infrastruktur (servrar, nätverk, lagring) med KOD istället för att klicka i gränssnitt.

Exempel – Istället för att klicka dig fram i Windows:

```
# Terraform-kod som skapar en fil
resource "local_file" "example" {
  content  = "Hej världen!"
  filename = "C:/temp/hello.txt"
}
```



Varför IaC?

Fördel	Förklaring
Konsekvent	Samma kod varje gång – samma resultat
Spårbart	Allt sparas i Git – full historik
Återanvändbart	Skapa 100 servrar lika enkelt som 1

Imperativt / PowerShell

Recipe card

1. Sätt ugnen på 175°
2. Smält smör
3. Vispa ägg...

Imperativt = Steg-för-steg-instruktioner.
"Gör exakt dessa steg."

→ Skapa en mapp, kopiera en fil,
starta en tjänst.

Deklarativt / Terraform



Deklarativt = Beskriv önskat slutresultat.
"Jag vill ha en färdig kaka, du löser hur."

→ Mappen SKA finnas,
Tjänsten SKA vara igång.

Vilket är bäst? Båda! PowerShell och Terraform kompletterar varandra i automationskedjan.

Imperativ (PowerShell) Deklarativ (Terraform)

Skapa en mapp	"Mappen SKA finnas"
Kopiera en fil	"Filen SKA finnas med rätt innehåll"
Starta en tjänst	"Tjänsten SKA vara igång"

Vilket är bäst? Båda! PowerShell (imperativ) och Terraform (deklarativ) kompletterar varandra.

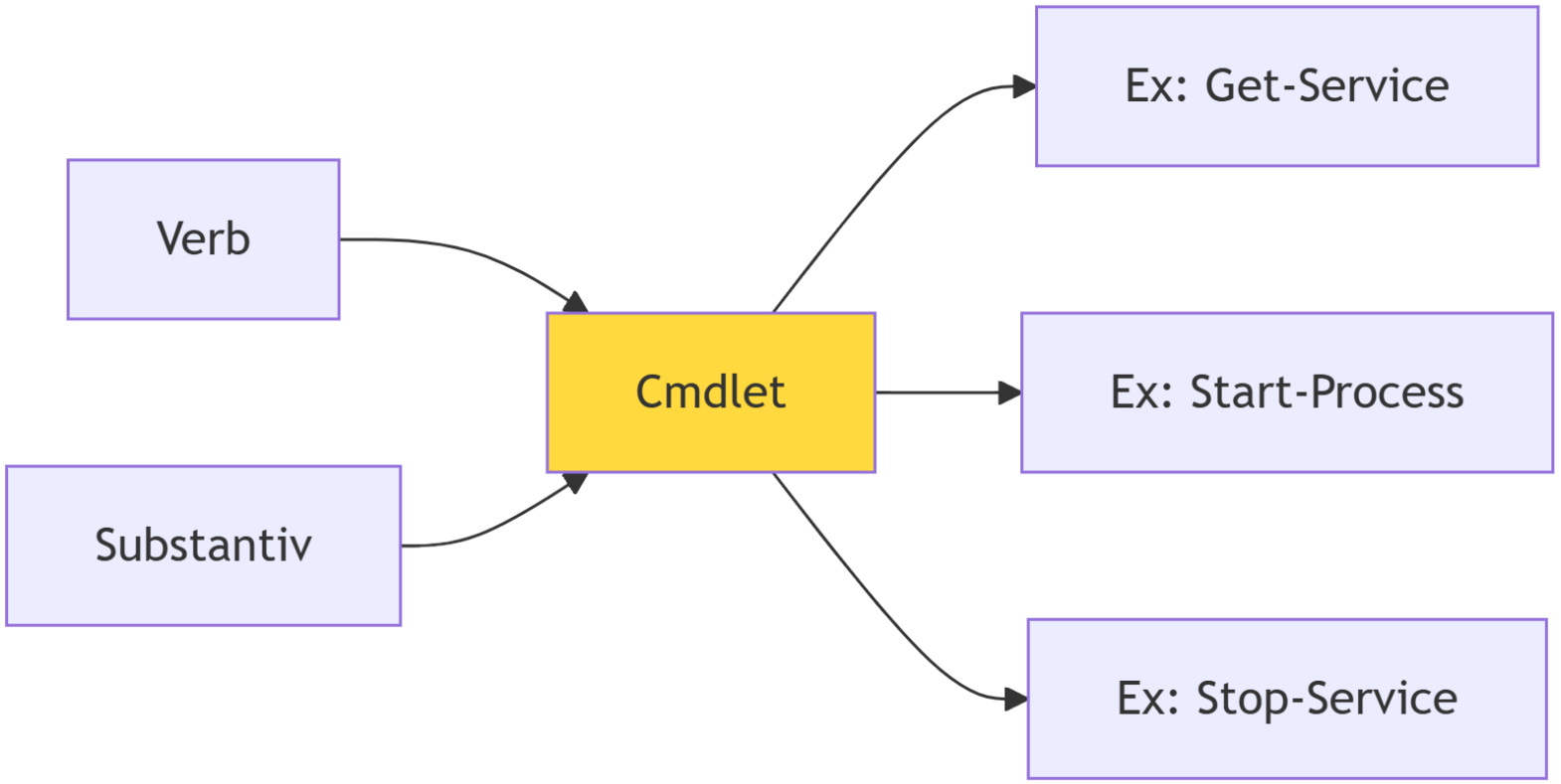
PowerShell vs. Bash

Egenskap	PowerShell	Bash
Plattform	Windows (samt Linux/Mac)	Linux/Mac (samt WSL2 i Windows)
Huvudfokus	Windows-administration	Linux-administration
Datahantering	**Objekt-baserad** (Kraftfullt!)	Text-baserad
Filändelse	`.ps1`	`.sh`

Huvudfokus i kursen är PowerShell för dess kraftfulla objekthantering, men grundläggande Bash krävs för att navigera Linux via WSL2.

PowerShell – Grundläggande Koncept

Cmdlets (Command-lets) – Grunden i PowerShell



Cmdlets har alltid formatet Verb-Substantiv:

Cmdlet	Vad gör den?
Get-Service	Hämta tjänster
Start-Process	Starta en process
Stop-Service	Stoppa en tjänst
New-Item	Skapa en ny fil/mapp
Set-Content	Skriv innehåll till en fil

Pipelining – Kedja ihop kommandon:
Get-Service | Where-Object {$_.Status -eq "Running"}
Detta hämtar alla tjänster och filtrerar fram de som är igång.
Varför är detta kraftfullt? Du kan kombinera enkla kommandon till komplexa operationer.

PowerShell 7.7 – Vad Är Nytt 2026?

Funktion	Vad betyder det för dig?
New-Guid v7	Bättre prestanda för unika ID:n
Bättre felhantering	Tydligare felmeddelanden
Förbättrad Tab-komplettering	Snabbare kodning
Snabbare modulinstallation	Installera flera moduler samtidigt

Varför är detta viktigt?

- PowerShell uppdateras kontinuerligt
- Microsoft satsar stort på PowerShell för automatisering
- PowerShell 7.7 fungerar på Windows, Linux och Mac
- **DSC (Desired State Configuration) i version 3** får nu även Python-stöd – vilket gör det ännu kraftfullare!

WSL2 – Windows Subsystem for Linux version 2

En funktion i Windows som låter dig köra Linux direkt i Windows – utan VirtualBox!

Aspekt	WSL2	VirtualBox
Prestanda	Snabbt (nära native)	Långsammare
Resursanvändning	Låg (dynamisk)	Hög (dedikerad)
Starttid	2–3 sekunder	30+ sekunder
Integration	Perfekt med Windows	Behöver extra konfiguration

Varför WSL2?

- Snabbare än VirtualBox för de flesta uppgifter
- Mindre resurskrävande – perfekt för äldre datorer
- Integrerar sömlöst med VS Code och Windows

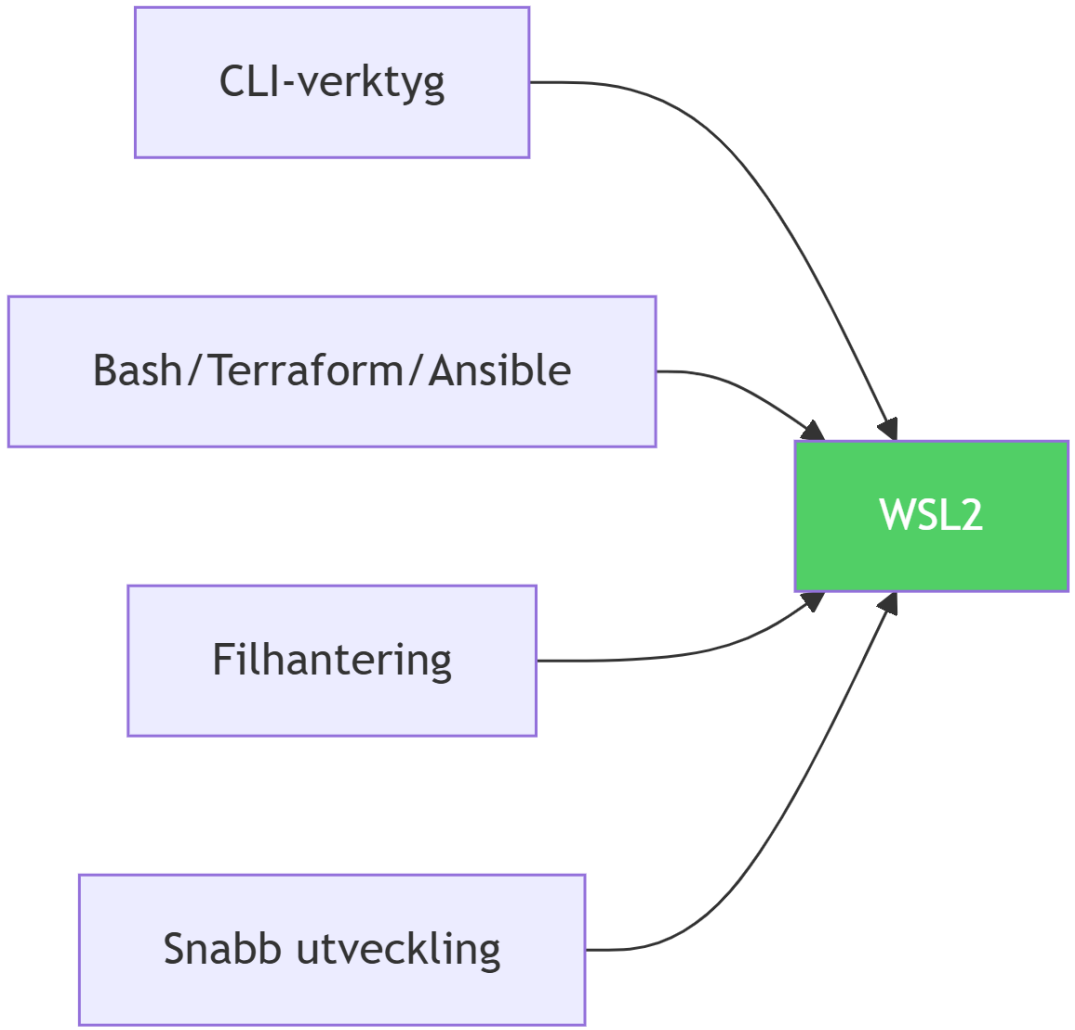
WSL2 vs VirtualBox – När Använder Man Vad?

	WSL2 ✅	VirtualBox ⚠️
Prestanda	Nära native (snabbt)	Långsammare
Resursanvändning	Låg (dynamisk)	Hög (dedikerad RAM krävs)
Starttid	2-3 sekunder	30+ sekunder
Integration	Sömlös Windows-integration	Isolerad (kräver konfiguration)

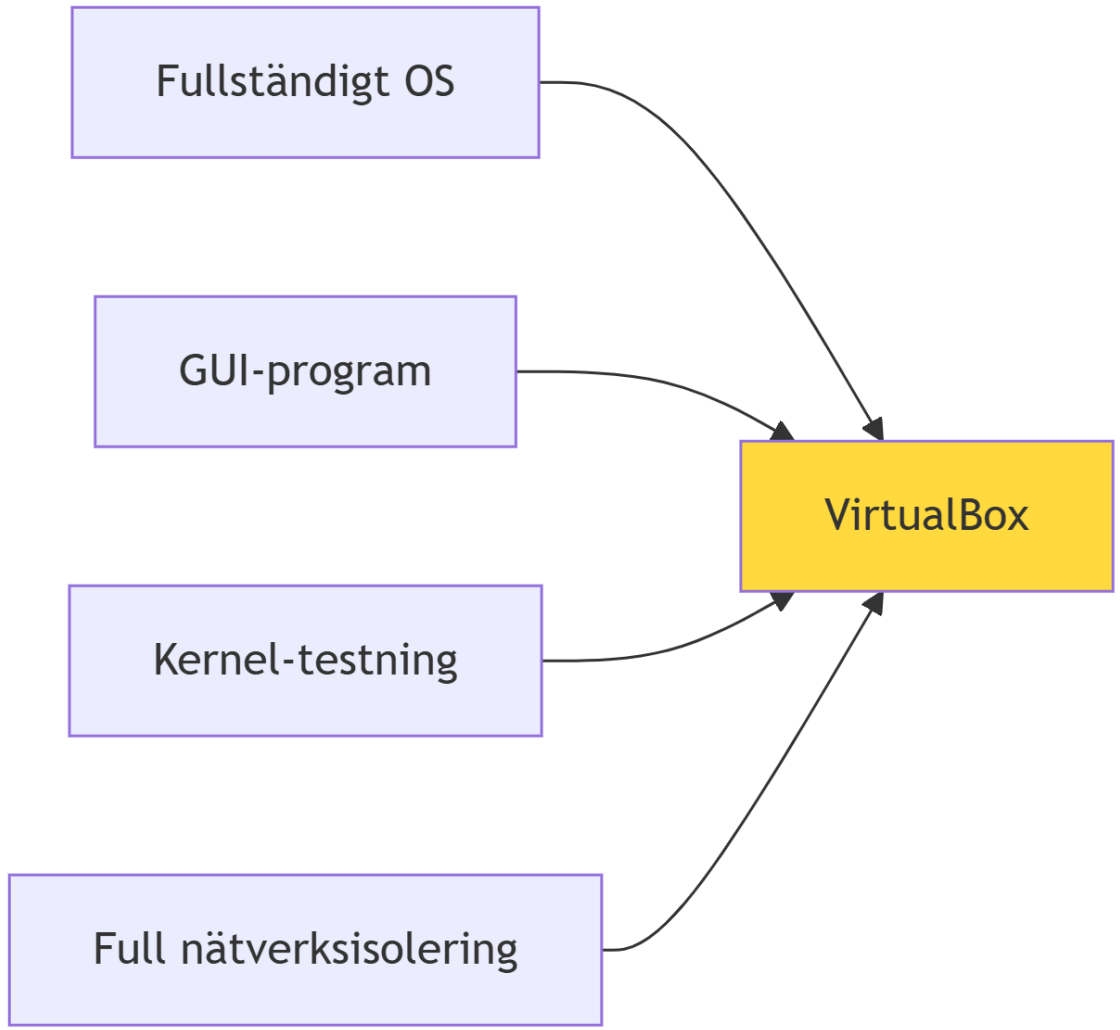
Rule of Thumb

WSL2 är den moderna standarden. Kör Linux direkt i Windows. (VirtualBox rekommenderas endast för specifika Mac-användare eller avancerad kernel-testning).

WSL2 (Vår huvudplattform)



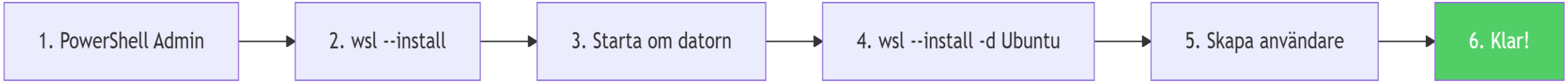
VirtualBox (Alternativ för Mac-studenter)



Viktigt: Om du har en äldre dator med begränsad RAM rekommenderas WSL2 starkt. VirtualBox kan vara tungt och långsamt på äldre hårdvara.

Installera WSL2 – Steg-för-Steg

Steg 1: Öppna PowerShell som Administratör



Detaljerade instruktioner:

1. Högerklicka på Start → "Windows PowerShell (Administratör)"
2. Skriv: wsl --install
3. Vänta medan Windows installerar WSL2
4. **Starta om datorn** (viktigt!)
5. Öppna PowerShell och skriv: wsl --install -d Ubuntu
6. Följ instruktionerna – skapa ett användarnamn och lösenord

Krav: Windows 10 version 2004 eller senare, eller Windows 11.

Verifiera WSL2-Installationen

- Öppna PowerShell
- Kör: `wsl -l -v`
- Se att VERSION står på 2
- `Windows PowerShell`
- `Copyright (C) Microsoft Corporation. All rights reserved.`
-
- `PS C:\Users\Användare> wsl -l -v`
- `  NAME      STATE           VERSION`
- `* Ubuntu    Running         2`
- **Om det inte fungerar:**
- Kontrollera att du startat om datorn efter installationen
- Kör: `wsl --set-default-version 2`
- Kontrollera att virtualisering är aktiverat i BIOS/UEFI

Vanliga PowerShell-Kommandon

- Din första övning: Skriv Get-Service i PowerShell – vad ser du?

Kommando	Vad gör det?	Exempel
Get-Command	Visa alla kommandon	Get-Command *Service*
Get-Help	Visa hjälp för ett kommando	Get-Help Get-Service
Get-Service	Visa alla tjänster	Get-Service -Name W3SVC
Get-Process	Visa alla processer	Get-Process notepad
Get-ChildItem	Lista filer i mappen	Get-ChildItem C:\temp
Get-Date	Visa dagens datum och tid	Get-Date -Format "yyyy-MM-dd"
Get-Location	Visa aktuell mapp	Get-Location (eller pwd)

Prova dessa kommandon i WSL2 (Ubuntu):

- Din första övning: Öppna WSL2 och skriv ls -la – vad ser du?

Kommando	Vad gör det?	Exempel
ls	Lista filer	ls -la
cd	Byt mapp	cd /tmp
pwd	Visa aktuell mapp	pwd
mkdir	Skapa mapp	mkdir mittprojekt
cp	Kopiera fil	cp fil.txt /tmp/
mv	Flytta/byt namn	mv fil.txt nyfil.txt
rm	Ta bort fil	rm fil.txt
cat	Visa filinnehåll	cat fil.txt
echo	Skriv ut text	echo "Hej världen!"
whoami	Visa användarnamn	whoami

PowerShell vs Bash – Samma Uppgift, Olika Syntax

- **Se skillnaderna – båda gör samma sak!** PowerShell är objekt-baserat, Bash är text-baserat. PowerShell är därför kraftfullare för Windows-administration.

Uppgift	PowerShell	Bash
Variabel	$namn = "Erik"	namn="Erik"
Skriv ut	Write-Host "Hej $namn"	echo "Hej $namn"
Lista filer	Get-ChildItem	ls -la
Skapa mapp	New-Item -Path "test" -ItemType Directory	mkdir test
Ta bort fil	Remove-Item fil.txt	rm fil.txt
Loop	for ($i=1; $i -le 5; $i++) { Write-Host $i }	for i in {1..5}; do echo $i; done
Villkor	if ($age -gt 18) { Write-Host "Myndig" }	if ["$age" -gt 18]; then echo "Myndig"; fi
Funktion	function Hej { Write-Host "Hej!" }	hej() { echo "Hej!"; }

Varför VS Code?

- Gratis och öppen källkod
- Fungerar med PowerShell, Python, Terraform, YAML
- Perfekt WSL2-integration
- Många användbara tillägg

Tillägg	Varför?
Remote – WSL	Öppna filer direkt i WSL2
PowerShell	Syntax-highlighting, IntelliSense
Terraform	Syntax-highlighting, autoformat
Python	Syntax-highlighting, felsökning
YAML	Syntax-highlighting för Ansible

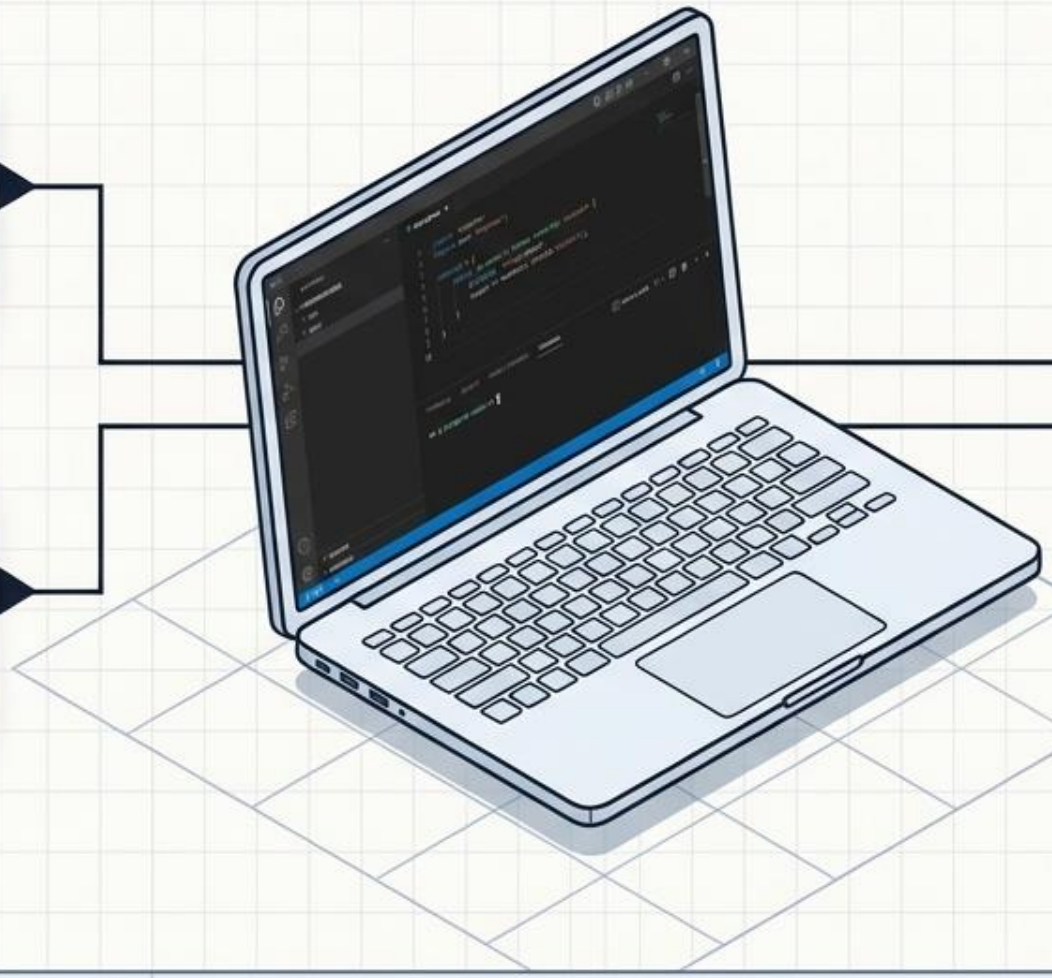
Din Setup: Visual Studio Code är navet för allt vi gör. Gratis, öppen källkod och integrerar alla automationsspråk.

Remote – WSL

Kritiskt: Öppna filer och terminaler direkt i WSL2.

PowerShell

Ger syntax-highlighting och IntelliSense.



Terraform

Autoformatering och validering av IaC.

Python

För avancerad felsökning och integration.

Installation:



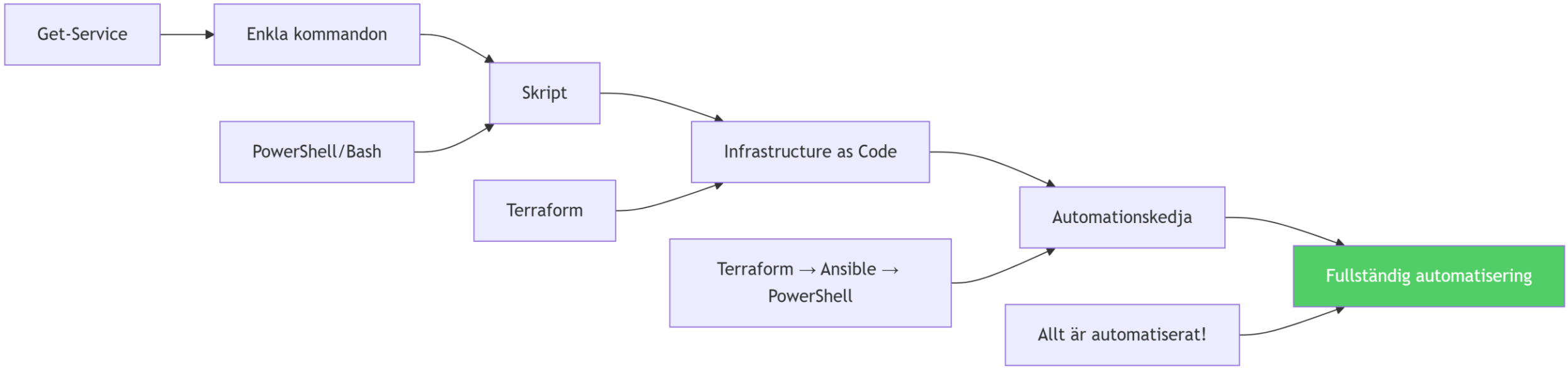
Ladda ner från:
https://code.visualstudio.com/



Installera tillägget: Remote – WSL



Öppna VS Code och klicka på "Open Folder in WSL"



Resan från enkla kommandon till full automatiserad miljö

Resurser och Verktyg – För Din Egen Inlärning

- **Microsofts Officiella Resurser:**
- - PowerShell dokumentation – Officiell dokumentation
- - WSL2 installationsguide – Steg-för-steg
- **Mobilappar:**
- - Termux (Android) – Kör Linux-kommandon på mobilen! Perfekt för att öva Bash när du är på språng.
- - iSH Shell (iOS) – Linux-liknande miljö på iPhone/iPad.
- - SoloLearn – PowerShell – Lär dig PowerShell på mobilen med interaktiva övningar.
- **Gratis Övningsmiljöer:**
- - WSL2 – Din Windows-dator (redan installerat!)
- - GitHub Codespaces – Webbaserad utvecklingsmiljö
- - Azure Cloud Shell – Gratis Linux-terminal i molnet

Workshop – Kom Igång Med Verktygen

- **Steg 1: Installera WSL2**
- wsl --install (PowerShell Admin) → wsl --install -d Ubuntu
- **Steg 2: Installera VS Code med Remote-WSL-tillägg**
- **Steg 3: Testa PowerShell**
- - Starta PowerShell 7: pwsh
- - Kör: Get-Service | Select-Object -First 5
- **Steg 4: Testa Bash i WSL2**
- - Starta WSL2: wsl
- - Kör: ls -la och whoami
- **Steg 5: Skapa din första PowerShell-variabel**
- $namn = "DittNamn"
- Write-Host "Hej $namn!"
- **Steg 6: Skapa din första Bash-variabel**
- namn="DittNamn"
- echo "Hej $namn!"

Gör en **quiz**

—

https://forms.gle/kCp73ogMPpfXgjmE6